

WORKING:**1. Signal Generation (Physical Layer)**

- **Input:** The user performs one of the **20 defined activities** (e.g., *Walking, Jumping, Falling*).
- **TENG Mechanism:** As the body moves, the Triboelectric Nanogenerator (attached to the joint/shoe) experiences **contact-separation** or **sliding**.
- **Output:** This mechanical energy is converted into high-voltage AC electrical spikes.
 - *Standing:* Negligible output (Noise ~10-15).
 - *Walking:* Rhythmic moderate pulses (Values ~300-600).
 - *Jumping/Falling:* Massive, sharp peaks (Values ~1023 maxed out).

2. Data Acquisition (Hardware Layer)

- **Circuit:** The AC signal from the TENG goes through a **Bridge Rectifier** (to convert AC to DC) and a **Voltage Divider** (to step down high voltage to 5V/3.3V).
- **Digitization:** The **Arduino** (or ESP32) reads the voltage at the Analog Pin (A0).
- **Sampling:** The microcontroller captures the voltage **50 times per second (50Hz)**.
- **Raw Data:** The analog voltage is converted into a digital integer between **0 and 1023**.

3. Data Preprocessing (Software Layer)

- **Windowing:** The Deep Learning model cannot predict activity from a single number (e.g., "500"). It needs a **Time Window**.
- **Segmentation:** The system groups the incoming stream of numbers into 2-second chunks (e.g., 100 data points).
- **Normalization:** The values (0-1023) are often scaled down to a range of 0-1 for faster processing by the neural network.

4. Pattern Recognition (Deep Learning Layer)

- **Model:** The **1D CNN (Convolutional Neural Network)** receives the 2-second window.
- **Feature Extraction:**
 - The CNN detects **frequency** (how fast the spikes happen).
 - It detects **amplitude** (how high the spikes are).
 - It detects **shape** (sharp spikes vs. smooth waves).
- **Comparison:** The model compares the live input pattern against the patterns learned from your **arduino_collected_data.csv** training set.

5. Classification & Output

- **Inference:** The model calculates a probability score for all 20 activities.

- **Decision:** It selects the activity with the highest probability.
 - *Example:* If the model sees high amplitude spikes at 3Hz, it classifies it as **"Running Fast"**.
 - *Example:* If it sees a sudden single max spike followed by zero, it classifies it as **"Falling"**.
- **Action:** The predicted label is displayed on a screen or sent via SOS (using the ESP32) if a critical activity like "Falling" is detected.

Phase 1: Data Collection System using TENG and Arduino:

1. Working Principle:

The system utilizes a Triboelectric Nanogenerator (TENG) as a self-powered active sensor.¹ When the user performs physical activities (like walking or running), the TENG layers undergo periodic contact and separation.² This mechanical friction generates surface static charges via triboelectrification, which drives a flow of electrons through the external circuit due to electrostatic induction, creating distinct AC voltage spikes unique to each movement.³

2. Hardware Interface:

The TENG sensor is connected to the Arduino Uno analog input pin (A0) via a load resistor circuit.

- **Voltage Sensing:** The Arduino's 10-bit ADC (Analog-to-Digital Converter) samples the TENG's electrical potential at a rate of **50Hz** (every 20ms).
- **Current Calculation:** Using Ohm's Law ($I = V / R$), the system computes the instantaneous current flowing through the circuit based on the fixed resistance ($R = 35\Omega$).
- **Charge Accumulation:** The transferred charge is derived ($Q = V \times I \times 10$) to quantify the energy magnitude of the impact.

3. Data Acquisition Process:

The Arduino processes the raw analog signals and transmits them serially to the computer in a structured CSV format. For every activity sample, the system logs five key parameters:

1. **Timestamp:** The precise timing of the signal to capture temporal patterns.
2. **Activity Label:** Ground truth annotation (e.g., "Walking," "Falling") for supervised learning.
3. **Voltage:** The amplitude of the signal representing impact force.
4. **Current:** The flow rate of electrons indicating signal intensity.
5. **Charge:** A derived feature representing the total electrical output of the motion.

CODE FOR COLLECTING DATA USING AURDINO UNO:

```
String activity = "Standing";

const int voltagePin = A0;

float R = 35.0;

void setup() {
  Serial.begin(115200);

  Serial.println("timestamp,activity,voltage,current,charge");
}

void loop() {
  unsigned long currentTime = millis();

  float timestamp = currentTime / 1000.0;

  float voltage = analogRead(voltagePin) * (5.0 / 1023.0);

  float current = voltage / R;

  float charge = voltage * current * 10.0;

  Serial.print(timestamp, 3);

  Serial.print(",");

  Serial.print(activity);

  Serial.print(",");

  Serial.print(voltage, 6);

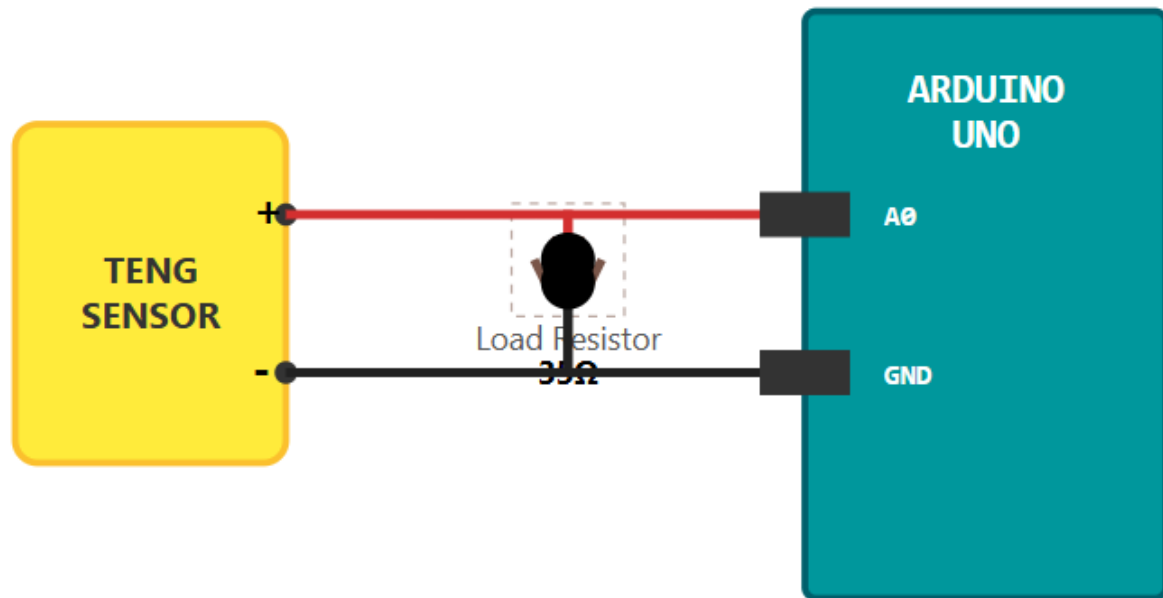
  Serial.print(",");

  Serial.print(current, 6);

  Serial.print(",");

  Serial.println(charge, 6);

  delay(20);
}
```

CIRCUIT DIAGRAM:**TENG Sensor Data Collection Circuit****FILE OF DATA COLLECTED:**
[LINK](#)
Phase 2: Deep Learning Model Development for Human Activity Recognition:

1. Objective The objective of this phase is to develop a robust **1D Convolutional Neural Network (1D-CNN)** capable of classifying 20 distinct human activities based on the electrical impulse patterns (Voltage, Current, Charge) collected from the TENG sensor in Phase 1.

2. Data Preprocessing Pipeline Before feeding the raw data from `teng_har_dataset_100.csv` into the neural network, it undergoes rigorous preprocessing to ensure the model learns temporal patterns rather than isolated values.

- **Feature Selection:** The model utilizes three input channels: **Voltage (V)**, **Current (I)**, and **Charge (Q)**. These parameters combined provide a complete profile of the mechanical energy intensity and duration.
- **Label Encoding:** The categorical labels (e.g., "Walking", "Jumping") are converted into numerical integers (0 to 19) using **Label Encoding** and subsequently transformed into **One-Hot Encoded** vectors (e.g., [0, 0, 1, 0...]) for multi-class classification.
- **Sliding Window Segmentation (Critical Step):**

Since a single data point (e.g., 0.02V) is insufficient to identify an action, the continuous data stream is sliced into **Time Windows**.

- **Window Size:** 100 samples (representing 2 seconds of data at 50Hz).
- **Overlap:** 50% (50 samples) to ensure continuity between windows and prevent data loss at the edges of movements.
- **Input Shape:** The final input tensor shape for the model is (N_samples, 100, 3).

3. Model Architecture: 1D Convolutional Neural Network (CNN) We utilize a 1D-CNN because it is highly effective at extracting local features (such as sharp peaks, rhythmic waves, or noise patterns) from time-series data, similar to how standard CNNs detect edges in images.

- **Input Layer:** Receives the 100 x 3 matrix (Time Steps × Features).
- **Convolutional Layers (Conv1D):**
 - *Layer 1:* 64 Filters, Kernel Size 3. Scans the electrical signals to detect low-level features like voltage spikes and signal rising edges.
 - *Layer 2:* 32 Filters. Combines low-level features to recognize complex patterns like the rhythmic cycle of "Cycling" or the double-impact of "Jumping Jacks."
- **Pooling Layers (MaxPooling1D):** Reduces the dimensionality of the data by keeping only the most prominent features (highest peaks), making the model computationally efficient and reducing overfitting.
- **Dropout Layers:** Randomly deactivates 20% of neurons during training to force the model to learn robust features and prevent it from memorizing the training data.
- **Fully Connected (Dense) Layers:**
 - *Flatten Layer:* Converts the 2D feature maps into a 1D vector.
 - *Output Layer:* A Dense layer with **20 neurons** (one for each activity) using the **Softmax** activation function to output a probability distribution (e.g., "95% probability it is Running").

4. Training Configuration The model is trained in a Jupyter Notebook environment using the TensorFlow/Keras framework.

- **Optimizer:** Adam (Adaptive Moment Estimation) is used for its efficiency in handling sparse gradients and noisy sensor data.
- **Loss Function:** Categorical Crossentropy is selected as the error metric since this is a multi-class classification problem (20 classes).
- **Batch Size:** 32 (The model updates weights after seeing 32 windows of data).
- **Epochs:** 20-50 (The dataset is passed through the network multiple times to minimize error).

5. Expected Outcome Upon completion of Phase 2, the trained model is saved as a .h5 file. This model file contains the mathematical "weights" required to recognize the 20 activities and will be ready for deployment in the final testing phase.

CODE FOR THE TRAINING OF CNN MODEL:

!pip install tensorflow pandas numpy scikit-learn matplotlib seaborn → use this to install necessary librarys

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from tensorflow.keras.utils import to_categorical

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Conv1D, MaxPooling1D, Flatten, Dropout

from sklearn.metrics import confusion_matrix

df = pd.read_csv('teng_har_dataset_100.csv')

WINDOW_SIZE = 100

STEP_SIZE = 50

encoder = LabelEncoder()

df['label_encoded'] = encoder.fit_transform(df['activity'])

classes = encoder.classes_

num_classes = len(classes)

scaler = StandardScaler()

df[['voltage', 'current', 'charge']] = scaler.fit_transform(df[['voltage', 'current', 'charge']])

X = []

y = []

for i in range(0, len(df) - WINDOW_SIZE, STEP_SIZE):

    window_data = df[['voltage', 'current', 'charge']].iloc[i:i+WINDOW_SIZE].values

    label = df['label_encoded'].iloc[i:i+WINDOW_SIZE].mode()[0]
```

```

X.append(window_data)

y.append(label)

X = np.array(X)

y = np.array(y)

y_encoded = to_categorical(y, num_classes=num_classes)

X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2,
random_state=42)

model = Sequential()

model.add(Conv1D(filters=64, kernel_size=3, activation='relu',
input_shape=(WINDOW_SIZE, 3)))

model.add(MaxPooling1D(pool_size=2))

model.add(Dropout(0.2))

model.add(Conv1D(filters=32, kernel_size=3, activation='relu'))

model.add(MaxPooling1D(pool_size=2))

model.add(Flatten())

model.add(Dense(50, activation='relu'))

model.add(Dense(num_classes, activation='softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=20, batch_size=16, validation_data=(X_test,
y_test))

plt.figure(figsize=(10, 5))

plt.plot(history.history['accuracy'], label='Training Accuracy')

plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title('Model Accuracy')

plt.xlabel('Epoch')

plt.ylabel('Accuracy')

plt.legend()

plt.show()

y_pred = model.predict(X_test)

y_pred_classes = np.argmax(y_pred, axis=1)

y_true = np.argmax(y_test, axis=1)

```

```
cm = confusion_matrix(y_true, y_pred_classes)

plt.figure(figsize=(12, 10))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes,
yticklabels=classes)

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.title('Confusion Matrix')

plt.show()

model.save('teng_har_model.h5')
```

Phase 3: Wearable Hardware Development & Real-Time Inference:

1. Objective

The final phase involves transitioning from a data collection prototype to a fully functional wearable device. The objective is to integrate the TENG sensor with a compact microcontroller (ESP32/Arduino Nano) and deploy the trained 1D-CNN model to perform real-time Human Activity Recognition (HAR).

2. Hardware Architecture

To ensure portability and wearability, the bulky Arduino Uno used in Phase 1 is replaced with a compact, battery-operated system.

A. Microcontroller Unit (MCU) - ESP32

We utilize the ESP32 module due to its high processing speed (240 MHz dual-core), built-in Bluetooth/Wi-Fi capabilities, and small form factor. It acts as the central processing unit to read sensor data and communicate results.

B. Signal Conditioning Circuit

Since TENG sensors generate high-voltage AC spikes that can damage microcontrollers, a conditioning circuit is integrated:

1. **Rectification:** A full-wave bridge rectifier converts the AC TENG signal into a DC pulse.
2. **Voltage Division:** A resistive divider network ensures the voltage remains within the safe logic level (0 - 3.3V) for the ESP32.
3. **Filtering:** A small capacitor (optional) smoothes high-frequency noise while preserving the distinct activity spikes.

C. Wearable Integration

The TENG sensor is embedded into a wearable form factor (e.g., a shoe insole or knee patch). The MCU and battery are housed in a 3D-printed enclosure attached to the user's ankle or waist to minimize motion interference.

3. Real-Time Inference Workflow

The system operates in a continuous loop to recognize activities instantly:

Step 1: Real-Time Data Acquisition

The ESP32 samples the TENG sensor voltage at 50Hz (matching the training phase). It computes the Current and Charge values locally and streams this 3-channel data (Voltage, Current, Charge) to the inference engine.

Step 2: Sliding Window Buffering

Unlike static image classification, HAR requires a time-series context. The system uses a Circular Buffer (First-In-First-Out queue) to maintain the latest 100 samples (2 seconds). As new data arrives, the oldest data is discarded, maintaining a valid window for prediction.

Step 3: Model Inference

The buffered data is fed into the trained 1D-CNN Model.

- Input: (1, 100, 3) Tensor.
- Processing: The model applies the learned weights and filters to the live data.
- Output: A probability array for the 20 activities (e.g., [0.01, 0.98, 0.01...]).

Step 4: Activity Classification

The system identifies the index with the highest probability (argmax). To prevent flickering predictions (e.g., switching rapidly between "Walking" and "Running"), a Smoothing Filter (Majority Vote) is applied over the last 5 predictions.

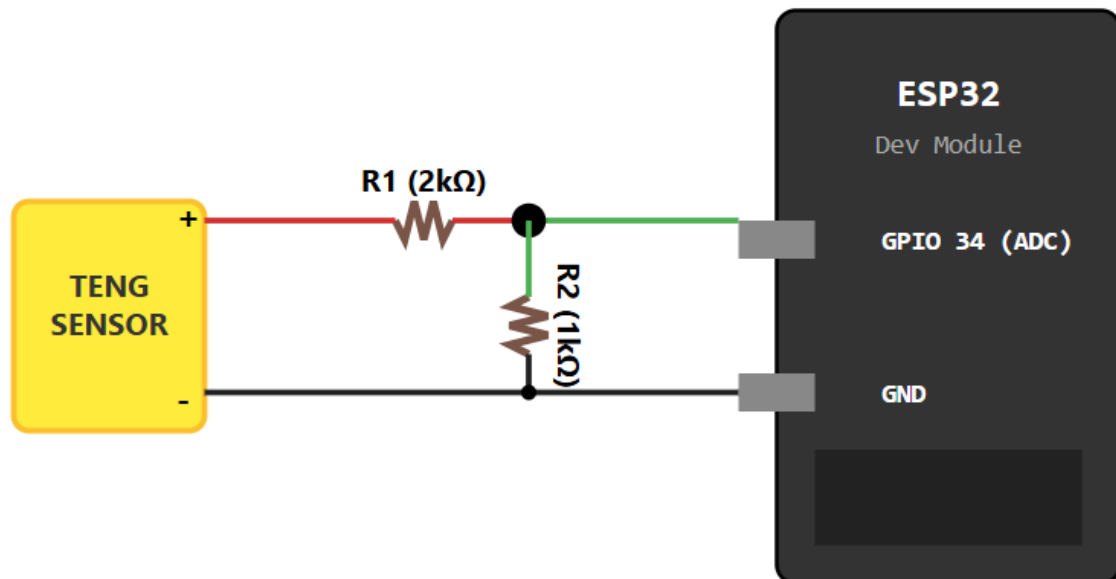
4. Output & Application

The recognized activity is:

1. Displayed on a local OLED screen (optional).
2. Transmitted via Bluetooth/Wi-Fi to a smartphone app or dashboard.
3. Used to trigger alerts (e.g., if "Falling" is detected, an SOS signal is activated).

CIRCUIT DIAGRAM:**Phase 3: ESP32 Wearable Data Circuit**

Adapts high-voltage TENG signals for safe 3.3V ESP32 processing

**CODE:**

```

import serial

import numpy as np

import pandas as pd

import time

from collections import deque

from tensorflow.keras.models import load_model

from sklearn.preprocessing import StandardScaler

SERIAL_PORT = 'COM3'

BAUD_RATE = 115200

MODEL_PATH = 'teng_har_model.h5'

DATASET_PATH = 'teng_har_dataset_100.csv'

WINDOW_SIZE = 100

LABELS = [

```

```

"Standing", "Sitting", "Lying Down", "Walking Slow", "Walking Normal", "Walking Fast",
"Running Slow", "Running Fast", "Jumping", "Jumping Jacks", "Stairs Up", "Stairs Down",
"Squatting", "Cycling", "Bending", "Falling", "Clapping", "Waving Hand", "Punching",
"Kicking"

```

```

]

```

```

print("Loading Model...")

```

```

model = load_model(MODEL_PATH)

```

```

print("Model Loaded!")

```

```

print("Loading Training Data for Calibration...")

```

```

df_train = pd.read_csv(DATASET_PATH)

```

```

scaler = StandardScaler()

```

```

scaler.fit(df_train[['voltage', 'current', 'charge']])

```

```

print("Calibration Complete!")

```

```

try:

```

```

    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)

```

```

    print(f"Connected to {SERIAL_PORT}")

```

```

except:

```

```

    print("Error: Connect your ESP32!")

```

```

    exit()

```

```

data_buffer = deque(maxlen=WINDOW_SIZE)

```

```

print("Starting Real-Time Recognition...")

```

```

print("-" * 30)

```

```

while True:

```

```

    try:

```

```

        line = ser.readline().decode('utf-8').strip()

```

```

        if not line or "timestamp" in line:

```

```

            continue

```

```

        parts = line.split(',')

```

```

        if len(parts) >= 5:

```

```

            voltage = float(parts[2])

```

```

current = float(parts[3])
charge = float(parts[4])
data_buffer.append([voltage, current, charge])
if len(data_buffer) == WINDOW_SIZE:
    input_data = np.array(data_buffer)
    input_data = scaler.transform(input_data)
    input_data = input_data.reshape(1, WINDOW_SIZE, 3)
    prediction = model.predict(input_data, verbose=0)
    predicted_index = np.argmax(prediction)
    confidence = np.max(prediction)
    activity_name = LABELS[predicted_index]
    if confidence > 0.6:
        print(f"Detected: {activity_name} ({confidence*100:.1f}%)")
        if activity_name == "Falling":
            print("!!! ALARM: FALL DETECTED !!!")
        else:
            print("Analyzing...")
except ValueError:
    pass
except KeyboardInterrupt:
    print("Stopping...")
    break

```

USAGE OF THE PROJECT:

The fabricated Triboelectric Nanogenerator (TENG) sensor functions as a self-powered active motion sensor designed for non-invasive biomechanical monitoring. When integrated into a wearable interface, such as a shoe insole or a knee joint patch, the device operates on the vertical contact-separation mode, converting the mechanical energy from human movement into distinct electrical signals. As the subject performs various daily activities like walking, running, or kicking, the rhythmic pressure and physical displacement induce surface triboelectric charges, generating unique voltage and current patterns that serve as the input data for the deep learning classification model.